

# Software Engineering

## Complete Exam Study Guide

---

|    |                                       |
|----|---------------------------------------|
| 01 | Process Framework Activities (CPMCD)  |
| 02 | Umbrella Activities                   |
| 03 | Software Myths                        |
| 04 | Process Models (Incremental & Spiral) |
| 05 | W■HH Project Management               |
| 06 | Function Points & COCOMO Model        |
| 07 | Risk Management & RMMM                |
| 08 | SRS & UML Diagrams                    |
| 09 | Software Testing                      |
| 10 | Maintenance, SCM & SQA                |

Based on lecture slides • All 10 topics covered

## 01 Process Framework Activities (CPMCD)

A process framework establishes the foundation for a complete software engineering process. It encompasses five core activities:

|                          |                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| <b>C — Communication</b> | Talk with customers/stakeholders to understand project requirements and define software features           |
| <b>P — Planning</b>      | Create Software Project Plan: defines workflow, technical tasks, risks, resources, product & work schedule |
| <b>M — Modeling</b>      | Create models to understand requirements; shows design of software to achieve requirements                 |
| <b>C — Construction</b>  | Code generation (manual or automated) + Testing to uncover errors in the code                              |
| <b>D — Deployment</b>    | Deliver software to customer → collect feedback based on evaluation → software support                     |

*Memory trick: **C**lever **P**rogrammers **M**ake **C**ool **D**eliveries*

### Key Points to Remember

- Construction = code generation AND testing (not just writing code)
- Deployment includes feedback collection and software support — not just delivery
- Each pass through the framework results in better understanding of the software

## 02 Umbrella Activities

Umbrella activities run in the **background throughout** the entire software development lifecycle. They help the team manage and control **progress, quality, change & risks**. They also establish a skeleton architecture for software engineering work.

|                                                  |                                                                                                                                      |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Software Project Tracking &amp; Control</b>   | Assess progress against the project plan; take any necessary action to maintain the schedule                                         |
| <b>Risk Management</b>                           | Evaluates (assesses) risks that may affect the outcome of the project or the quality of the product                                  |
| <b>Software Quality Assurance (SQA)</b>          | Defines and conducts the activities required to ensure software quality                                                              |
| <b>Technical Reviews</b>                         | Assesses software engineering work products to uncover and remove errors before they propagate to the next activity                  |
| <b>Measurement</b>                               | Defines and collects process, project and product measures that assist the team in delivering software that meets stakeholders needs |
| <b>Software Configuration Management (SCM)</b>   | Manages the effects of change throughout the software process                                                                        |
| <b>Reusability Management</b>                    | Defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components    |
| <b>Work Product Preparation &amp; Production</b> | Encompasses the activities required to create work products such as models, documents, logs, forms and lists                         |

*Key distinction: Framework activities (CPMCD) are sequential phases; umbrella activities run continuously alongside them.*

## 03 Software Myths

**Definition:** Misleading attitudes about software and the process used to build it that cause serious problems. Three categories: Management, Customer, Practitioner (Developer).

### Management Myths

|                |                                                                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MYTH</b>    | We have standards and procedures to build a system — that's enough.                                                                                                                        |
| <b>REALITY</b> | Are practitioners aware of them? Do they reflect modern SE practice? Are they complete? In many cases — NO.                                                                                |
| <b>MYTH</b>    | We have the newest computers and development tools.                                                                                                                                        |
| <b>REALITY</b> | It takes much more than the latest hardware. Computer-Aided Software Engineering (CASE) tools are more important than hardware.                                                            |
| <b>MYTH</b>    | We can add more programmers and catch up the schedule.                                                                                                                                     |
| <b>REALITY</b> | Fred Brooks: 'Adding people to a late software project makes it later.' Existing team must spend time educating newcomers. People can be added only in a planned, well-coordinated manner. |
| <b>MYTH</b>    | I outsourced development, so I can relax and wait for the final product.                                                                                                                   |
| <b>REALITY</b> | If an organisation doesn't understand how to manage and control software projects internally, it will invariably struggle when it outsources.                                              |

### Customer Myths

|                |                                                                                                                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MYTH</b>    | A general statement of objectives (requirements) is sufficient to start development.                                                                                                                              |
| <b>REALITY</b> | Comprehensive statements aren't always possible, but an ambiguous statement of objectives can lead to disaster. Clear requirements require effective and continuous communication between customer and developer. |
| <b>MYTH</b>    | Requirement changes can be easily accommodated because software is very flexible.                                                                                                                                 |
| <b>REALITY</b> | Software requirements do change, but the impact of change varies with the time it is introduced. Early changes cost little; late changes cost much more.                                                          |

### Practitioner (Developer) Myths

|                |                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MYTH</b>    | Once we write the program, our job is done.                                                                                                                                                                                    |
| <b>REALITY</b> | Industry data: 60–80% of all effort on software is expended AFTER it is delivered to the customer for the first time. Experts say 'the sooner you begin writing code, the longer it takes to get done.'                        |
| <b>MYTH</b>    | I can't assess quality until the program is running.                                                                                                                                                                           |
| <b>REALITY</b> | One of the most effective software quality assurance mechanisms — the technical review — can be applied from the beginning of a project. Software reviews are more effective quality filters than testing for finding defects. |
| <b>MYTH</b>    | A working program is the only deliverable work product.                                                                                                                                                                        |
| <b>REALITY</b> | A working program is only one part of a software configuration. Models, documents, and plans provide a foundation for successful engineering and guidance for software support.                                                |
| <b>MYTH</b>    | Software engineering is about unnecessary documentation.                                                                                                                                                                       |

**REALITY**

Software engineering is not about creating documents — it is about creating a quality product. Better quality leads to reduced rework, and reduced rework results in faster delivery times.

## 04 Process Models — Incremental & Spiral

### Incremental Process Model

Combines elements of **linear** and **parallel** process flows. Applies linear sequence in an iterative manner. An initial core working product is delivered first, then each linear sequence produces deliverable 'increments' of the software.

**Example — Word-processing software:** Increment 1: basic file management & editing | Increment 2: sophisticated editing | Increment 3: spelling & grammar checking | Increment 4: advanced page layout

**When to use:** Requirements clearly defined and understood, but staffing unavailable for complete implementation by the business deadline.

| Advantages                                                                                                                                                                                                                                                                                                                | Disadvantages / Limitations                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Working software generated quickly and early</li><li>• Easier to test and debug during smaller iteration</li><li>• Customer can respond to each built</li><li>• Lowers initial delivery cost</li><li>• Easier to manage risk (risky pieces identified during iteration)</li></ul> | <ul style="list-style-type: none"><li>• Requires clear initial requirements</li><li>• Not suitable when full scope is unknown</li><li>• Integration issues may arise between increments</li></ul> |

### The Spiral Model

Provides the potential for **rapid development**. Software is developed in a series of evolutionary releases. Early iteration releases may be prototypes; later iterations provide more complete versions. Divided into framework activities (CPMCD) — each activity represents one segment of the spiral.

|                                   |                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------|
| <b>Each pass through Planning</b> | Results in adjustments to the project plan and cost & schedule based on feedback |
| <b>Early iterations</b>           | May produce prototypes                                                           |
| <b>Later iterations</b>           | Provide increasingly more complete versions of the software                      |

**When to use:**

|                                                                                    |                                                                                               |                                                                                    |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Large scale / high-risk projects</li></ul> | <ul style="list-style-type: none"><li>• When costs and risk evaluation is important</li></ul> | <ul style="list-style-type: none"><li>• Users are unsure of their needs</li></ul>  |
| <ul style="list-style-type: none"><li>• Requirements are complex</li></ul>         | <ul style="list-style-type: none"><li>• New product line</li></ul>                            | <ul style="list-style-type: none"><li>• Significant changes are expected</li></ul> |

| Advantages                                                                                                                                                                                                                                          | Disadvantages                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• High risk avoidance (enhanced)</li><li>• Strong approval and documentation control</li><li>• Additional functionality added at later date</li><li>• Software produced early in software lifecycle</li></ul> | <ul style="list-style-type: none"><li>• Can be a costly model to use</li><li>• Risk analysis requires highly specific expertise</li><li>• Project success is highly dependent on risk analysis phase</li><li>• Doesn't work well for smaller projects</li></ul> |

## 05 W■HH Principle — Project Management

Boehm suggests the W■HH approach — applicable **regardless of the size or complexity** of the software project. Addresses: project objectives, milestones, schedules, responsibilities, management/technical approaches, and required resources.

|                  |                                                                                                                                                                               |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Why?</b>      | Why is the system being developed? Enables all parties to assess the validity of business reasons — does the business purpose justify expenditure of people, time, and money? |
| <b>What?</b>     | What will be done? Answers help the team establish a project schedule by identifying key project tasks and milestones required by the customer                                |
| <b>When?</b>     | When will it be accomplished? Project schedule to achieve each milestone                                                                                                      |
| <b>Who?</b>      | Who is responsible? Defines the role and responsibility of each team member                                                                                                   |
| <b>Where?</b>    | Where are they organizationally located? Customer, end user and other stakeholders also have responsibility                                                                   |
| <b>How?</b>      | How will the job be done technically and managerially? Management and technical strategy must be defined                                                                      |
| <b>How much?</b> | How much of each resource is needed? Develop estimation for budget, people, and time                                                                                          |

## 06 Function Points & COCOMO Model

### Function-Oriented Metrics — Function Point (FP)

Uses a measure of the **functionality delivered** by the application as a normalisation value. Most widely used metric of this type.

$$FP = \text{Count Total} \times [ 0.65 + 0.01 \times \text{Sum}(Fi) ]$$

**Count Total** = sum of all weighted FP entries

**Fi (i=1..14)** = complexity Value Adjustment Factors (VAF)

| Information Domain Value        | Count | Simple | Average | Complex |
|---------------------------------|-------|--------|---------|---------|
| External Inputs (EIs)           | —     | 3      | 4       | 6       |
| External Outputs (EOs)          | —     | 4      | 5       | 7       |
| External Inquiries (EQs)        | —     | 3      | 4       | 6       |
| Internal Logical Files (ILFs)   | —     | 7      | 10      | 15      |
| External Interface Files (EIFs) | —     | 5      | 7       | 10      |
| Count Total                     |       |        |         | —       |

| FP Advantages                                                                                                                                                                                    | FP Disadvantages                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Programming language independent</li><li>• Based on data more likely known in early stages</li><li>• More attractive as an estimation approach</li></ul> | <ul style="list-style-type: none"><li>• Requires subjective data ('sleight of hand')</li><li>• Counts of information domain can be difficult to collect</li><li>• Has no direct physical meaning — just a number</li></ul> |

### COCOMO Model — Constructive Cost Estimation

Proposed by **Boehm**. Software cost estimation done through three stages: **Basic COCOMO** → **Intermediate COCOMO** → **Complete COCOMO**.

#### Basic COCOMO

$$\text{Effort} = a1 + (\text{KLOC})^{a2} \text{ PM}$$

$$\text{Tdev} = b1 \times (\text{Effort})^{b2} \text{ Months}$$

**KLOC** = estimated size in Kilo Lines of Code

**PM** = person-months (total effort)

**Tdev** = time to develop (months)

| Project Type | a1  | a2   | b1  | b2   |
|--------------|-----|------|-----|------|
| Organic      | 2.4 | 1.05 | 2.5 | 0.38 |
| Semidetached | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded     | 3.6 | 1.20 | 2.5 | 0.32 |

### Worked Example

**Given:** Organic type, 32,000 lines of code, average salary = Rs. 15,000/month

$$\text{Effort} = 2.4 + (32)^{1.05} \text{ PM} = 91 \text{ PM}$$

$$\text{Tdev} = 2.5 \times (91)^{0.38} \text{ Months} = 14 \text{ Months}$$

$$\text{Cost} = 14 \times 15,000 = \text{Rs. } 2,10,000$$

### Nominal Estimate Warning

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>Faster than nominal</b> | Cost increases drastically if you try to complete the project in less than the estimated duration  |
| <b>Slower than nominal</b> | Completing over a longer period than estimated results in almost no decrease in the estimated cost |

### Intermediate & Complete COCOMO

|                            |                                                                                                                                                                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Intermediate COCOMO</b> | Refines basic estimate using 15 cost drivers (multipliers) based on attributes of software development — Product, Computer, Personnel, and Development Environment. Project manager rates each parameter on scale 1–3.                        |
| <b>Complete COCOMO</b>     | Addresses shortcoming of basic/intermediate: they treat software as a single homogeneous entity. Complete COCOMO considers each subsystem separately (organic/semidetached/embedded) and estimates the sum. This reduces the margin of error. |

## 07 Risk Management & RMMM

A risk is a potential (probable) problem — which might happen and might not.

|                    |                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------|
| <b>Uncertainty</b> | The risk may or may not happen — there are no 100% risks (those are called constraints) |
| <b>Loss</b>        | If the risk becomes a reality, unwanted consequences or losses occur                    |

*Risk concerns future happenings · involves change in mind, opinion, actions, places · involves choice and the uncertainty that choice entails*

### Risk Categorisation — Approach 1

|                        |                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Project risks</b>   | Threaten the project plan. If they become real, it is likely the project schedule will slip and costs will increase            |
| <b>Technical risks</b> | Threaten the quality and timeliness of the software to be produced. If real, implementation may become difficult or impossible |
| <b>Business risks</b>  | Threaten the feasibility of the software to be built. If real, they threaten the project or the product                        |

*Sub-categories of Business risks: Market risk · Strategic risk · Sales risk · Management risk · Budget risk*

### Risk Categorisation — Approach 2

|                            |                                                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Known risks</b>         | Can be uncovered after careful evaluation of the project plan, the business/technical environment, and other reliable information sources (e.g., unrealistic delivery date) |
| <b>Predictable risks</b>   | Deduced (draw conclusion) from past project experience (e.g., past staff turnover)                                                                                          |
| <b>Unpredictable risks</b> | Can and do occur, but are extremely difficult to identify in advance                                                                                                        |

### RMMM — Risk Mitigation, Monitoring & Management

|                        |                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Risk Mitigation</b> | Problem avoidance activity (avoidance). Strategy to avoid risk before it occurs.                                    |
| <b>Risk Monitoring</b> | Project tracking activity. Track factors that provide indication of whether a risk is becoming more or less likely. |
| <b>Risk Management</b> | Includes contingency plans for what happens if risk occurs. Executed when mitigation fails.                         |

### RMMM Plan

- Documents all work performed as part of risk analysis
- Used by the project manager as part of the overall project plan
- Some teams use a Risk Information Sheet (RIS) per individual risk instead of a formal plan
- RIS is maintained using a database system for easy creation, ordering, and searching

## 08 SRS, Class Diagrams & Activity Diagrams

### SRS — Software Requirements Specification

A document that **completely describes what the proposed software should do** without describing how it will do it. Also helps clients understand their own needs.

#### SRS Contains:

|                                                   |                                   |                                              |
|---------------------------------------------------|-----------------------------------|----------------------------------------------|
| • Complete information description                | • Detailed functional description | • A representation of system behaviour       |
| • Performance requirements and design constraints | • Appropriate validation criteria | • Other information suitable to requirements |

#### Characteristics of a Good SRS:

|                                        |                                                                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Correct</b>                         | All user requirements are stated in the requirements document. No specified tool to assure correctness.                       |
| <b>Unambiguous</b>                     | Every stated requirement has only one interpretation                                                                          |
| <b>Complete</b>                        | Requirements clearly define what the software is required to do                                                               |
| <b>Ranked for Importance/Stability</b> | Each requirement identified to make differences. Stability implies probability of changes in future.                          |
| <b>Modifiable</b>                      | Requirements document created so that changes can be made easily                                                              |
| <b>Traceable</b>                       | Source of each requirement is clear and facilitates reference in future                                                       |
| <b>Verifiable</b>                      | Specified requirements can be verified with a cost-effective process to check whether final software meets those requirements |
| <b>Consistent</b>                      | Subsets of individual requirements do not conflict with each other                                                            |

#### Problems Without SRS:

- System would not be properly implemented according to customer needs
- Software developers would not know whether what they are developing is exactly what the customer requires
- Very difficult for maintenance engineers to understand the functionality of the system
- Very difficult for user document writers to write user manuals properly

### Class Diagram

Used to describe objects in systems and different types of relationships between them. Used to construct and visualise object-oriented systems. A class is a **blueprint of an object**.

|                   |                                                                                                            |
|-------------------|------------------------------------------------------------------------------------------------------------|
| <b>Class Name</b> | Bold, centred, capital letters, meaningful. Abstract class name in italic format.                          |
| <b>Attributes</b> | Second compartment. Syntax: accessModifier attributeName:dataType=defaultValue. First character lowercase. |
| <b>Operations</b> | Third compartment. Syntax: accessModifier methodName(argumentList):returnType. First character lowercase.  |

#### Access Modifiers:

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| <b>+ Public</b>  | Accessible by all classes, whether in same package or another |
| <b>- Private</b> | Cannot be accessed outside the enclosing/declaring class      |

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| <b># Protected</b>         | Can be accessed only by subclasses and within a class |
| <b>~ Package</b>           | Accessible by all classes within the package only     |
| <b>Underlined = Static</b> | Member can be accessed using class name only          |

## Activity Diagram & Swimlane Diagram

An activity diagram visually presents a **series of operations or flow of control** in a system — similar to an algorithm or flowchart. Shows 'How' the system works. Most useful during early stages of designing algorithms and workflows.

|                          |                                                                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Activity</b>          | Elongated ovals. Unlabelled arrow from one to another indicates first must complete before second begins.                                                           |
| <b>Branch (Decision)</b> | Diamond shape. One incoming arrow, two or more outgoing arrows each with condition in [square brackets].                                                            |
| <b>Initiation</b>        | Solid circle with an outgoing arrow = starting point of the activity diagram.                                                                                       |
| <b>Termination</b>       | Bull's eye — solid circle surrounded by hollow circle. Only incoming arrows. Execution ends here.                                                                   |
| <b>Fork</b>              | Synchronisation bar (heavy line): splits one activity into several concurrent activities                                                                            |
| <b>Merge</b>             | Synchronisation bar: combines several concurrent activities back into a single activity                                                                             |
| <b>Swimlane</b>          | Divides activity diagram into columns showing which person/department is responsible for each activity. Lines across boundaries = interactions between departments. |

## 09 Software Testing

### Three Views of Test Objects

| Black Box Testing<br>(Close Box / Specification-Based)                                                                                                | White Box Testing<br>(Open Box / Structural)                                                                                                                     | Grey Box Testing                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• No knowledge of internals</li><li>• Testing based only on specification</li><li>• Testing as a user</li></ul> | <ul style="list-style-type: none"><li>• Internals fully known to tester</li><li>• Testing based on actual source code</li><li>• Testing as a developer</li></ul> | <ul style="list-style-type: none"><li>• Partial knowledge of source code</li><li>• Internals relevant to testing known</li></ul> |

### Black Box Testing (Specification-Based)

Tester has access only to running code and the specification it is supposed to satisfy. Test cases are written with no knowledge of internal workings. Emphasis is only on ensuring that the contract is met.

| Advantages                                                                                                                                                                                                                                                                                                                                                               | Disadvantages                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Scalable — not dependent on size of code</li><li>• No knowledge of implementation needed</li><li>• Tester and developer can be truly independent</li><li>• Tests done with requirements in mind</li><li>• Does not excuse inconsistencies in specification</li><li>• Test cases can be developed in parallel with code</li></ul> | <ul style="list-style-type: none"><li>• Must examine pre-condition, identify equivalence classes</li><li>• All possible inputs such that all classes are covered</li><li>• Apply specification to input to write expected output</li><li>• May leave many program paths untested</li><li>• Weighting of program paths is not possible</li></ul> |

### White Box Testing (Structural)

Internal structure/design/implementation of the module being tested is **known to the tester**. Focus on ensuring that even abnormal invocations are handled gracefully. Goal: ensure all statements and conditions have been executed at least once.

**White-box derives test cases that:**

- Guarantee all independent paths within a module exercised at least once
- Exercise all logical decisions on their true and false sides
- Execute all loops at their boundaries
- Exercise internal data structures to ensure their validity

*Applicable levels: Unit Testing (paths within a unit) · Integration Testing (paths between units) · System Testing (paths between subsystems)*

### White-Box Testing Strategies

|                                          |                                                                                                                                                                                                                                      |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Statement Coverage</b>             | Aims to design test cases so every statement in a program is executed at least once. If a statement is not executed, it's very hard to determine if an error exists.                                                                 |
| <b>2. Branch Coverage (Edge Testing)</b> | Test cases designed to make each branch condition assume true and false values in turn. Each edge of the program's control flow graph is traversed at least once. Branch coverage GUARANTEES statement coverage (stronger strategy). |
| <b>3. Path Coverage</b>                  | Test cases executed so every path is executed at least once. All possible control paths taken, including all loop paths (zero times, once, multiple). Uses Flow Graph, Cyclomatic Complexity, and Graph Metrics. STRONGEST strategy. |

**Strength order: Statement coverage < Branch coverage < Path coverage**

## 10 Maintenance, Re-Engineering, SCM & SQA

### Types of Software Maintenance

|                               |                                                                                                                                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Perfective Maintenance</b> | Modifications to keep software usable over long period. Includes new features, new user requirements, improved reliability/performance, and changing functionalities as per user's changing needs. |
| <b>Preventive Maintenance</b> | Modifications to prevent future problems. Addresses issues not significant now but may cause serious problems later. Comprises documentation updating, code optimisation, and code restructuring.  |

### Re-Engineering, Reverse Engineering & Forward Engineering

|                                |                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Software Re-Engineering</b> | Update software to keep it current WITHOUT impacting functionality. Design is changed and programs re-written. Legacy software cannot keep tuning with the latest technology. Example: UNIX was re-engineered from assembly to C.                                                                                                                                                      |
| <b>Reverse Engineering</b>     | Extracts design information from source code. Abstraction level should be as high as possible. The process should be capable of deriving: procedural design representations, program/data structure info, object models, data flow models, entity relationship models. Directionality is one-way — all extracted info is provided to the software engineer.                            |
| <b>Forward Engineering</b>     | Process of obtaining desired software FROM the specifications brought by reverse engineering. Same as software engineering process, BUT always carried out AFTER reverse engineering. Does not simply create a modern equivalent of an older program — new user and technology requirements are integrated. The redeveloped program extends the capabilities of the older application. |

### Software Configuration Management (SCM)

A set of activities developed to **manage change throughout the software life cycle**. Purpose: systematically control changes to the configuration and maintain the integrity and traceability throughout the system's life cycle.

|                    |                                                                              |
|--------------------|------------------------------------------------------------------------------|
| <b>Objective 1</b> | Identify all items that collectively define the software configuration       |
| <b>Objective 2</b> | Manage changes to one or more of these items                                 |
| <b>Objective 3</b> | Facilitate the construction of different versions of an application          |
| <b>Objective 4</b> | Ensure software quality is maintained as the configuration evolves over time |

*SCM Layers (inside out): SCIs → Identification → Change control → Version control → Configuration auditing → Reporting*

|                                          |                                                                                                                                                                                       |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SCI (Software Configuration Item)</b> | Flows outward through SCM layers throughout its useful life. Assigned to a specific version (version control). A record of the SCI is maintained for configuration auditing purposes. |
| <b>Change Control Layer</b>              | Does not apply if no changes are requested for the SCI                                                                                                                                |

### Software Quality — SQA vs SQC

**Quality:** Developed product meets its specification. **Quality Management:** Ensuring the required level of quality is achieved.

| Criteria | SQA — Software Quality Assurance | SQC — Software Quality Control |
|----------|----------------------------------|--------------------------------|
|----------|----------------------------------|--------------------------------|

|                    |                                                                          |                                                             |
|--------------------|--------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Definition</b>  | Set of activities for ensuring quality in software engineering PROCESSES | Set of activities for ensuring quality in software PRODUCTS |
| <b>Focus</b>       | Process focused                                                          | Product focused                                             |
| <b>Orientation</b> | Prevention oriented                                                      | Detection oriented                                          |
| <b>Breadth</b>     | Organisation wide                                                        | Product/project specific                                    |
| <b>Scope</b>       | Relates to all products created by a process                             | Relates to specific product                                 |
| <b>Activities</b>  | Process Definition, Implementation, Audits, Training                     | Reviews, Testing                                            |

## FTR — Formal Technical Review

A software **quality control activity** performed by software engineers (and others).

1. Uncover errors in function, logic, or implementation for any representation of the software
2. Verify that the software under review meets its requirements
3. Ensure the software has been represented according to predefined standards
4. Achieve software that is developed in a uniform manner
5. Make projects more manageable

*During the FTR, a reviewer (the recorder) actively records all issues raised. These are summarised at the end of the review meeting, and a formal technical review summary report is completed.*

## Quick Revision — Must-Know Comparisons

| Topic                                 | Key Distinction                                                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Incremental vs Spiral</b>          | Incremental = linear + parallel, delivers working increments. Spiral = evolutionary releases with risk analysis each loop. |
| <b>SQA vs SQC</b>                     | SQA = process-focused, prevention. SQC = product-focused, detection (reviews and testing).                                 |
| <b>Reverse vs Forward Engineering</b> | Reverse = extracts design FROM code. Forward = builds software FROM extracted design/specs.                                |
| <b>Black vs White Box</b>             | Black box = specification only, no code access. White box = full code access, structural testing.                          |
| <b>Statement vs Branch vs Path</b>    | Path coverage is strongest; branch guarantees statement; statement is weakest.                                             |
| <b>Basic vs Intermediate COCOMO</b>   | Basic = product size only. Intermediate = adds 15 cost drivers for more accuracy.                                          |